Dental Clinic Management System with AI Cephalometric Analysis

Faculty of Graduate Studies for Statistical Research

Cairo University

Software Engineering

Submitted by:

Ahmed Mohamed Bakry

Fouad AbdelRahman Mohamed Ahmed

Rania Medhat Mohamed Ibrahim

Under supervision of: Dr. Maged Mamdouh

Cairo, June 2026

Updated from the current DentalCare repository implementation on 18 June 2026.

Document Control

Document name: Graduation Project.pdf Source reviewed: DentalCare repository at D:/Ai_Ceph_Project/DentalCare Review date: 18 June 2026 Current implementation status: ASP.NET Core MVC clinic system integrated with a FastAPI cephalometric AI service.

This document updates the graduation project report to match the current project implementation. It replaces older descriptions that referred to unsupported infrastructure with the actual modules, controllers, APIs, data entities, workflows, and verification status found in the repository.

Table of Contents

1. Introduction

2. Project Scope and Purpose

3. Current System Overview

4. Architecture

5. Functional Requirements

6. Non-Functional Requirements

7. Data Model

8. AI Cephalometric Analysis Module

9. DentalCare Web Application Module

10. API Contracts and Integration Flow

11. User Workflows

12. Testing and Verification

13. Project Review Findings

14. Limitations and Future Work

15. Conclusion

1. Introduction

DentalCare is a clinic management and AI-assisted orthodontic analysis project. The system combines a web-based clinic workflow with a Python AI service for lateral cephalometric X-ray analysis. The clinic application manages users, patients, appointments, and medical history. The AI service detects anatomical landmarks, calculates cephalometric measurements, classifies skeletal and vertical patterns, suggests treatment directions, generates visual overlays, and produces explainable decision support.

The project is intended for internal clinic use. Patients do not directly operate the system. Staff members manage patient registration and appointment scheduling, while doctors review patients, complete diagnoses, run AI analysis, approve final reports, and save results to the patient record.

Cephalometric analysis is clinically important because orthodontic diagnosis depends on accurate anatomical landmarks and reliable measurement interpretation. Manual tracing can be slow and inconsistent. The project therefore introduces AI-assisted landmark detection and structured measurement calculation while keeping the doctor responsible for final review and approval.

2. Project Scope and Purpose

The scope of the current project includes:

- Secure web authentication using ASP.NET Core Identity.
- Role-based access for Doctor and Staff users.
- Patient registration, search, editing, and patient profile views.
- Appointment creation, editing, filtering, cancellation, and completion tracking.
- Medical record creation and historical review.
- Synchronization of DentalCare patients with the AI service patient registry.
- AI-assisted cephalometric X-ray analysis from the doctor workstation.
- Manual landmark review, drag editing, local refinement, and recalculation.
- Auto-calibration support from image ruler ticks.
- Protocol-based measurement calculation for supported cephalometric analyses.
- Diagnostic classification, treatment suggestion, visual overlay generation, and explainable AI.
- Doctor review gate before saving reports or exporting PDFs.

The purpose is to reduce manual administrative effort in a dental clinic and support orthodontic decision-making with repeatable, reviewable AI output.

3. Current System Overview

The repository contains two main applications:

1. DentalCare ASP.NET Core MVC application

- Location: DentalCare/ - Framework: .NET 8, ASP.NET Core MVC, Entity Framework Core, ASP.NET Core Identity - Database: SQL Server through DentalDbContext - Main responsibility: clinic operations, authenticated UI, patient and appointment management, medical records, and AI report approval.

2. AI cephalometric service

- Location: ai_service/ - Framework: FastAPI, Streamlit, PyTorch, OpenCV, Pillow, NumPy, ReportLab - Main responsibility: cephalometric landmark detection, measurement analysis, treatment planning support, patient-friendly explanations, visual overlays, and local case storage.

The root run_all.bat launcher starts both services:

- FastAPI service from ai_service using python api/main.py.
- DentalCare web app from DentalCare using dotnet watch run.

The default service URLs are:

- AI service: http://localhost:8000
- DentalCare web app: http://localhost:5192

4. Architecture

The project uses a two-service architecture.

```
Doctor or Staff User
        |
        v
ASP.NET Core MVC Web App
        |
        |  HTTP/JSON and multipart requests
        v
FastAPI Cephalometric AI Service
        |
        +-- HRNet landmark model
        +-- Measurement and diagnostic engines
        +-- Treatment suggestion engine
        +-- Overlay and report generation helpers
        +-- SQLite case and patient registry
```

4.1 ASP.NET Core MVC Layer

The MVC application contains controllers for:

- AccountController: login, registration, logout.
- PatientController: patient CRUD, patient details, Ceph patient synchronization.
- AppointmentController: staff appointment creation and editing.
- StaffController: staff dashboard, appointment filtering, cancellation.
- DoctorController: doctor dashboard, patient profile, diagnosis, AI analysis workflow, saving AI reports, exporting analysis PDFs.

Business and integration services include:

- IRepository<T> and Repository<T> for reusable Entity Framework access.
- CephIntegrationService for all HTTP communication with the FastAPI service.
- AiAnalysisPdfBuilder for per-patient AI report export from the doctor workstation.

4.2 FastAPI AI Layer

The AI service exposes both legacy composite endpoints and newer fine-grained endpoints. The DentalCare web app currently uses the newer staged AI endpoints for most analysis actions.

Main API modules include:

- api/main.py: FastAPI app, route definitions, startup model loading, server entry point.
- api/model.py: HRNet loading and inference wrappers.
- api/utils.py: preprocessing, postprocessing, landmark refinement, anatomical validation.
- api/analysis.py: geometric calculations and protocol measurement report building.
- api/measurements.py: Monte Carlo uncertainty summaries.
- api/measurement_analysis.py: quality checks, confidence intervals, refinement suggestions.
- api/diagnostic_engine.py: skeletal, vertical, dental, and craniofacial pattern diagnosis.
- api/treatment_engine.py: treatment option generation.
- api/ai_engine.py: OpenAI/Gemini narrative generation with local fallback text.
- api/calibration.py and api/calibration_auto.py: manual and automatic scale calibration.
- api/drawing.py: overlay and report graphics.
- api/repository.py: SQLite case and AI patient registry.

- api/schemas.py: Pydantic API contracts.

5. Functional Requirements

5.1 Authentication and Authorization

- The system shall allow users to register and log in.
- The system shall assign users to roles during registration.
- The system shall protect clinical pages using authorization attributes.
- Staff users shall manage patients and appointments.
- Doctor users shall perform diagnoses, run AI analysis, review outputs, and save clinical records.

5.2 Patient Management

- Staff and doctors shall view and search patient records.
- Staff shall create new patients.
- Doctors, staff, and admins shall edit patient information.
- The system shall store patient name, age, gender, phone, email, appointments, medical history, and optional CephPatientId.
- The system shall attempt to synchronize new patients to the AI service registry.

5.3 Appointment Management

- Staff shall create appointments for selected patients.
- Staff shall assign appointments to doctors.
- Staff shall edit appointments that are not completed and not in the past.
- Staff shall cancel appointments.
- Doctors shall see their daily appointments and completion status.
- Adding a diagnosis for a current patient visit shall mark the related appointment as completed when applicable.

5.4 Medical Records

- Doctors shall add diagnosis records to patient history.
- Patient profiles shall display medical history ordered by date.
- Saved AI analysis reports shall become MedicalRecord entries with structured notes.

5.5 AI Cephalometric Analysis

- Doctors shall upload X-ray images in JPEG, PNG, BMP, or TIFF format.
- The system shall detect cephalometric landmarks using the AI service.
- The system shall calculate measurements using the selected protocol.
- The system shall classify skeletal class and vertical pattern.
- The system shall generate treatment suggestions.
- The system shall display landmarks on an interactive canvas.
- Doctors shall manually move landmarks when needed.
- The system shall recalculate results after edited landmarks are submitted.
- Doctors shall refine landmarks through the AI service local image feature algorithm.
- Doctors shall auto-calibrate pixel-to-mm scale when ruler ticks are detected.

- The system shall generate explainable AI output for treatment and diagnosis decisions.
- The system shall generate patient-friendly explanation text when requested.
- The system shall require doctor review before saving or exporting the AI report.

6. Non-Functional Requirements

6.1 Security

- Use ASP.NET Core Identity for authentication.
- Restrict sensitive clinical functions with role-based authorization.
- Do not allow patient-facing direct access to internal clinic tools.
- Require doctor approval before AI output becomes part of the medical record.

6.2 Reliability

- The DentalCare app shall not crash if the AI service is unavailable.
- CephIntegrationService handles AI failures by returning null or controlled errors.
- SQL Server connection is configured with retry-on-failure behavior.
- Database migrations are applied on startup.

6.3 Usability

- Staff dashboards support search, date filtering, doctor filtering, and status counts.
- Doctor dashboards support daily appointment workflow.
- The AI analysis workstation includes upload, preview, protocol selection, calibration, result tabs, editable landmarks, review notes, save, and export.

6.4 Maintainability

- MVC controllers separate UI workflows by user role.
- Repository pattern centralizes basic data access.
- AI service logic is separated into focused modules for schemas, analysis, diagnosis, treatment, calibration, drawing, and persistence.
- The updated report now has an editable Markdown source to make future report revisions easier.

6.5 Clinical Safety

- AI output is assistive and not final by itself.
- The doctor review gate prevents unreviewed AI output from being saved or exported.
- Measurement rows include normal values, differences, statuses, labels, and interpretations when available.
- Explainable AI exposes decision chain, key drivers, uncertainty factors, and alternative interpretation.

7. Data Model

7.1 DentalCare SQL Server Entities

Patient:

- Id
- Name
- Age
- Gender
- Phone
- Email
- CephPatientId
- Appointments
- MedicalHistory

Appointment:

- Id
- Date
- CreatedAt
- Type
- Status
- PatientId
- DoctorId
- Patient

MedicalRecord:

- Id
- PatientId
- Patient
- Date
- VisitType
- Diagnosis
- Notes
- DoctorId

Identity tables are inherited from ASP.NET Core Identity through DentalDbContext, which extends IdentityDbContext.

7.2 AI Service SQLite Entities

The AI service repository module stores:

- AI patient registry records synchronized from DentalCare.
- Saved analysis cases containing patient identifier, age, sex, status, comment, scale, ethnic profile, filename, landmarks JSON, analysis JSON, and timestamps.

This local repository supports AI-side case workflows and export packaging.

8. AI Cephalometric Analysis Module

8.1 Landmark Model

The AI service is designed to load an HRNet model from:

```
ai_service/models/best_model.pth
```

On startup, api/main.py checks for this file. If the file is not present, the service starts with a warning, but model-based landmark detection endpoints cannot complete until weights are available.

The current implementation uses a 19-landmark working set mapped through shared.landmarks and converted between numeric IDs and named API response dictionaries.

8.2 Image Processing

The AI pipeline includes:

- Base64 or multipart image intake.
- Preprocessing to prepare images for inference.
- Heatmap and offset postprocessing.
- Landmark coordinate conversion back to image space.
- Optional anatomical shape validation.
- Optional local landmark refinement using image intensity or edge features.

8.3 Measurement Engine

The core measurement engine calculates supported cephalometric measurements from landmark coordinates. Current measurements include:

- SNA
- SNB
- ANB
- FMA (FH-MP)
- Facial Angle (N-S-Gn)
- SN-GoGn
- IMPA
- FMIA
- Interincisal angle
- Lower anterior facial height
- Nasolabial angle
- Articular angle (S-Ar-Go)
- Gonial angle (Ar-Go-Me)
- Posterior face height / Anterior face height ratio
- Sum of angles

Measurements are grouped into skeletal, vertical, dental, soft tissue, and skeletal pattern categories.

8.4 Protocols

The current supported protocols are:

Protocol ID	Name	Purpose
core_lateral	Core lateral cephalometric screening	Core skeletal and vertical screening
steiner	Steiner analysis	Skeletal, vertical, and incisor screening
eastman_basic	Eastman basic	Minimal sagittal screening
eastman	Eastman analysis	Skeletal and vertical subset
abo_american	ABO American Board screening	ABO/Steiner-style norms
tweed	Tweed triangle	FMA, IMPA, FMIA
downs	Downs vertical screening	Mandibular and interincisal screening
mcnamara	McNamara screening	Face height and nasolabial subset
jarabak	Jarabak analysis	Face-height ratio and cranial base angles
vertical_basic	Vertical pattern basic	Vertical and facial-axis screening

Each protocol defines required landmark IDs and computable measurements.

8.5 Diagnosis and Treatment Support

Diagnosis is generated from measurement rows and includes:

- Skeletal class
- Vertical pattern
- Severity and confidence
- Findings and recommendations
- Clinical notes
- Skeletal differential probabilities

Treatment suggestions are generated from the diagnostic report and patient age. Returned treatment items include treatment name, type, description, rationale, risks, duration, confidence, evidence metadata, retention recommendation, and predicted outcomes.

8.6 Explainable AI and Overlays

The explainability endpoint returns:

- Decision chain
- Key drivers
- Uncertainty factors
- Clinical confidence
- Alternative interpretation

The overlay endpoint can generate:

- X-ray tracing with landmarks
- X-ray tracing with measurements
- Tracing-only image
- Wiggle chart
- Measurement table
- Clinical cephalometric report image

9. DentalCare Web Application Module

9.1 Account Module

AccountController supports login, registration, role assignment, and logout. Password rules are configured in Program.cs and require digit, uppercase, lowercase, non-alphanumeric character, and minimum length of 8.

9.2 Staff Module

Staff users can:

- View dashboard metrics for selected dates.
- Filter by patient search text, date, doctor, and completion state.
- Create appointments.
- Edit valid appointments.
- Cancel appointments.
- Create patient records.

9.3 Doctor Module

Doctor users can:

- View daily appointment dashboard.
- Search patients in the daily appointment list.
- Open patient profiles.
- Add diagnoses and complete appointments.
- Run AI cephalometric analysis.
- Review and edit landmarks.
- Save approved AI analysis to patient history.
- Export approved AI analysis PDF reports.

9.4 AI Workstation UI

The doctor analysis page includes:

- Patient selector with age and sex auto-fill.
- X-ray file drop zone and preview.
- Analysis settings: age, sex, ethnic profile, protocol, px-to-mm scale.
- Auto-calibration button.
- Run analysis, recalculate, and refine buttons.
- Canvas viewer with zoom, pan, landmark drag editing, labels, planes, and image toggles.
- Result tabs for workspace, clinical summary, measurements, treatment plan, explainable AI, and exports.
- Review gate with checkbox and optional clinician notes.
- Save to patient and export PDF actions unlocked only after review.

10. API Contracts and Integration Flow

10.1 Health and Patient Sync

- GET /health
- POST /api/integration/patient

DentalCare uses these endpoints to confirm service availability and store or update an AI-side patient identity.

10.2 Main AI Analysis Flow

```
Doctor uploads X-ray in DentalCare
        |
        v
DentalCare /Doctor/UploadXray
        |
        v
CephIntegrationService.AnalyzeXrayAsync
        |
        +-- POST /ai/detect-landmarks
        +-- POST /ai/calculate-measurements
        +-- POST /ai/classify-diagnosis
        +-- POST /ai/suggest-treatment
        +-- POST /ai/generate-overlays
        |
        v
JSON result returned to analysis workstation
```

10.3 Doctor-Reviewed Recalculation Flow

```
Doctor edits landmarks on canvas
        |
        v
POST /Doctor/RecalculateAnalysis
        |
        v
CephIntegrationService.RecalculateFromLandmarksAsync
        |
        +-- POST /ai/calculate-measurements
        +-- POST /ai/classify-diagnosis
        +-- POST /ai/suggest-treatment
        +-- POST /ai/generate-overlays
        |
        v
Updated diagnosis and treatment result
```

10.4 Supporting AI Endpoints

- POST /auto-calibrate: detects px-to-mm scale from ruler ticks.
- POST /refine: snaps landmark coordinates to local image features.
- POST /patient-letter: generates patient-friendly explanation text.
- POST /ai/explain-decision: generates explainable AI reasoning.
- GET /protocols: returns supported clinical protocols.
- GET /ai/norms: returns reference norms metadata.

10.5 Legacy Compatibility Endpoints

The AI service still exposes:

- POST /predict
- POST /analyze
- POST /measurements

- POST /measurement-analysis
- POST /diagnose
- POST /ai-interpret
- POST /treatment-plan
- POST /export
- CRUD endpoints under /cases

These provide compatibility and standalone AI workflows.

11. User Workflows

11.1 Staff Workflow

1. Staff logs in.
2. Staff registers or updates patient data.
3. Staff creates an appointment and assigns a doctor.
4. Staff monitors the dashboard by date, doctor, and status.
5. Staff can cancel appointments when required.

11.2 Doctor Standard Visit Workflow

1. Doctor logs in.
2. Doctor views today's appointments.
3. Doctor opens a patient record.
4. Doctor adds diagnosis and notes.
5. System saves a MedicalRecord entry.
6. System marks the active appointment as completed when applicable.

11.3 AI Cephalometric Workflow

1. Doctor opens the AI analysis page.
2. Doctor selects a patient.
3. Doctor uploads a lateral cephalometric X-ray.
4. Doctor selects protocol, ethnic profile, age, sex, and scale.
5. Doctor runs AI analysis.
6. System detects landmarks and calculates measurements.
7. System returns diagnosis, confidence, warnings, clinical notes, treatment options, and overlay image.
8. Doctor reviews landmarks on the canvas.
9. Doctor may drag landmarks, refine landmarks, or auto-calibrate the scale.
10. Doctor recalculates after changes.
11. Doctor opens explainable AI when decision rationale is needed.
12. Doctor checks the review confirmation and enters notes.
13. Doctor saves the report to patient history or exports a patient PDF.

12. Testing and Verification

12.1 Verification Performed During This Review

Command:

```
dotnet build DentalCare/DentalCare.csproj --no-restore
```

Result:

- Build succeeded.
- 0 errors.
- 21 warnings.

The warnings are mostly nullable-reference warnings in models, controllers, and Razor views. There is also an unused field warning for PatientController._httpClient.

12.2 Current Automated Test State

The Python service contains pytest configuration, but the tests directory currently contains no active test files. The working tree also shows a deleted test file named ai_service/tests/test_growth_stage.py. Therefore, Python automated test coverage is currently not available from the active repository state.

An attempted pytest run from the current ai_service virtual environment also failed because pytest is not installed in that environment. This confirms that the repository needs both restored test files and test dependency setup before Python automated validation can be used.

12.3 Recommended Test Cases

Core MVC tests:

- Register user with Doctor role.
- Register user with Staff role.
- Create patient as staff.
- Create appointment as staff.
- Prevent editing completed or past appointments.
- Doctor dashboard shows only assigned appointments for doctor users.
- Add diagnosis creates a MedicalRecord and completes the appointment.

AI integration tests:

- /health returns status ok.
- Patient sync returns a stable AI patient ID.
- UploadXray rejects missing and unsupported files.
- UploadXray handles unavailable AI service gracefully.
- RecalculateAnalysis rejects empty landmark lists.
- SaveAnalysisReport rejects unreviewed AI results.
- ExportAnalysisPdf rejects unreviewed AI results.
- AutoCalibrate handles images without ruler ticks.
- RefineLandmarks returns accepted or rejected movement metadata.

AI service tests:

- Protocol validation returns missing landmark IDs correctly.

- Measurement calculation returns expected SNA, SNB, ANB, FMA, and IMPA for known landmarks.
- Diagnostic classification changes class when ANB is outside normal range.
- Treatment suggestion changes plan by skeletal class and age.
- Explain-decision returns decision chain and key drivers.

13. Project Review Findings

13.1 Strengths

- The project has a clear two-service architecture.
- The MVC app and FastAPI service communicate through explicit HTTP contracts.
- Role-based workflows are implemented with ASP.NET Core authorization.
- The doctor AI workflow includes a review gate before saving or exporting results.
- The AI service is modular and separates schemas, analysis, diagnosis, treatment, calibration, and rendering.
- The current .NET application builds successfully.
- The AI workflow supports manual correction, recalculation, explainability, and PDF export.

13.2 Important Observations

- Some README content is stale and still references files or claims not present in the current working tree, such as growth_stage.py, Docker, OpenTelemetry, Prometheus/Grafana, and a 44+ test suite.
- The active tests directory does not contain runnable pytest tests.
- The project currently depends on ai_service/models/best_model.pth for live landmark detection; without that file, model-based endpoints will return errors.
- There are nullable-reference warnings in the .NET project that should be cleaned up before production use.
- PatientController declares an unused _httpClient field.
- Several source files contain mojibake/encoding artifacts in comments or README text. This does not block compilation but should be cleaned for presentation quality.
- Some clinical AI responses are generated from rules and defaults when complete measurement evidence is unavailable. The report should describe the AI as assistive decision support, not an autonomous diagnostic device.

13.3 Working Tree State During Review

The repository contained existing modified files before this report update, including controller, service, view, and AI-service changes. This update did not revert those changes. The report describes the current working tree behavior observed during review.

14. Limitations and Future Work

Current limitations:

- No active Python test suite is present in the repository.
- Model weights are required but not included in the reviewed file list.
- Some project documentation is stale.
- Several .NET nullable warnings remain.
- AI analysis requires doctor review and should not be used as standalone medical diagnosis.
- The AI service can run without loading the model file, but landmark detection will fail until weights are installed.

Future improvements:

- Add unit and integration tests for both .NET and FastAPI modules.
- Restore or replace the deleted growth-stage test coverage if growth staging remains in scope.
- Add health checks that report model-loaded status, not only API availability.
- Normalize documentation encoding to UTF-8.
- Add CI build and test workflow.
- Add structured logging and audit trails for AI report approval.
- Add stronger validation for uploaded image size, dimensions, and content.
- Add clinician override fields for final diagnosis and treatment.
- Add model performance documentation with dataset, train/validation split, accuracy metrics, and known failure modes.
- Add deployment documentation for SQL Server, FastAPI, model weights, environment variables, and service startup.

15. Conclusion

The current DentalCare project is a functional graduation project that combines clinic management with AI-assisted cephalometric analysis. The ASP.NET Core application provides role-based patient, appointment, and record workflows. The FastAPI service provides a clinical AI pipeline for landmark detection, measurements, diagnosis, treatment suggestions, overlays, explanation, and exports.

The most important design decision is that AI output remains doctor-reviewed. The system supports automation, but the doctor remains responsible for final clinical approval before saving the result to patient history or exporting a report. This makes the project more appropriate for a clinical decision-support context than a fully automated diagnosis context.

Based on the current implementation, the project is ready to be presented as an integrated clinic management and AI cephalometric analysis system, with clear recommendations to improve test coverage, documentation accuracy, model deployment details, and code warning cleanup before production deployment.